RK3572 MSMART User Guide

文件标识：RK-KF-YF-C69

发布版本：V1.0.0

日期：2026-01-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文提供 RK3572 VOP MSMART 功能说明与使用指导。

产品版本

芯片名称	内核版本
RK3572	LINUX Kernel 6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	张玉炳	2026-01-23	初始版本

目录

RK3572 MSMART User Guide

1. 概述
2. MSMART Feature
3. MSMART 使用约束
   - 3.1 图层格式对齐要求
   - 3.2 图层 act_width 最小长度要求
   - 3.3 多宫格使用约束
4. 驱动配置说明
   - 4.1 单图层模式
   - 4.2 多宫格模式
   - 4.3 驱动配置注意事项
5. 多宫格模式功能说明
   - 5.1 多宫格空间布局说明
   - 5.2 多宫格图层格式说明
   - 5.3 多宫格缩放和mirror说明
6. 用户空间接口说明

1. 概述

MSMART 图层主要为 NVR 应用场景引入的一种新的图层，支持单图层和多宫格两种工作模式。下文将详细介绍 MSMART 图层的功能特性和使用要求，以及用户空间接口说明。

2. MSMART Feature

图层支持的格式：

RGB: ARGB8888、RGB888、RGB565、RGBA1010102、RGBA5551

YUV: YUV420(8,10bit)、YUV422(8,10bit)、YUV444(8,10bit)、YUV400(8,10bit)

MSMART 图层用于支持显示多宫格，有两种工作模式，单图层模式和多宫格模式(显示区域超过 1 个为多宫格)。

RK3572 上 MSMART0 支持 4K, 8 x 8 的显示多宫格，即最多 64 个宫格，横向最多只能 8 个宫格，纵向最多 64 个宫格。

RK3572 上 MSMART1 支持 2K, 6 x 6 的显示多宫格，即最多 64 个宫格，横向最多只能 6 个宫格，纵向最多 36 个宫格。

Note: 所有的 RGB 和 YUV 图层格式均不支持压缩和 TILE.

3. MSMART 使用约束

3.1 图层格式对齐要求

	YUV422	YUV420	YUV444_10bit	YUV422_10bit	YUV420_10bit	Other
act_width	2pixel	2pixel	2pixel	2pixel	2pixel	\
act_height	\	2line	\	\	2line	\
act_xoffset	2pixel	2pixel	4pixel	4pixel	4pixel	\
act_yoffset	\	2line	\	\	2line	\

3.2 图层 act_width 最小长度要求

format	act_width
ARGB8888/RGBA1010102	>=16
RGB888	>=16
RGB565/RGBA5551	>=16
YUV_8bit：YUV_444/YUV422/YUV420/YUV400	>=17
YUV_10bit：YUV_444/YUV422/YUV420/YUV400	>=16

3.3 多宫格使用约束

1. msmart 图层内的多宫格在空间上不能叠加;
2. msmart0 同一行至多只能 8 个显示宫格存在，msmart1 同一行至多只能有 6 个显示宫格存在;
3. msmart0 最多只能有 64 个宫格，msmart1 最多只能有 36 个宫格;
4. msmart 多宫格所有图层格式要保持相同, 不支持设置不同的图层格式；
5. msmart 多宫格不支持每个宫格独立缩放，只支持整体缩放，缩小系数要小于 2，放大系数限制和单图层模式一样(不超过 8 倍)；
6. msmart 不支持 x mirror, 支持整体 y mirror。

4. 驱动配置说明

4.1 单图层模式

单图层模式下，通过 AHB 总线配置图层信息到相关的寄存器，相关的寄存器具体如下表：

Registers	description
WIN_CTRL(0x00)	\
WIN_ACT_INFO(0x1c)	win_act_width[12:0] win_act_height[28:16]
WIN_DSP_INFO(0x20)	win_dsp_width[12:0] win_dsp_height[28:16]
WIN_DSP_ST(0x24)	win_dsp_xst[12:0] win_dsp_yst[28:16]
WIN_GRID0_YRGB_MST(0x50)	grid0_yrgb_mst[31:0]
WIN_GRID0_CBCR_MST(0x54)	grid0_cbcr_mst[31:0]
WIN_GRID0_VIR(0x58)	grid0_vir_stride[15:0] grid0_vir_stride_uv[31:16]
WIN_GRID0_ACT_INFO(0x5C)	grid0_act_width[12:0] grid0_act_height[28:16]
	grid0_act_xoffset[1:0]

Registers	description
WIN_GRID0_ACT_OFFSET(0x60)	grid0_act_xoffset[1:0] grid0_act_yoffset[17:16]
WIN_GRID0_DSP_OFFSET(0x64)	grid0_dsp_xoff[12:0] grid0_dsp_yoff[28:16]

WIN_ACT_INFO 和 WIN_GRID0_ACT_INFO 的配置保持一致；

WIN_GRID0_ACT_OFFSET 配置设置为 0；

WIN_GRID0_DSP_OFFSET 配置设置为 0；

WIN_DSP_ST 为实际在屏幕上的偏移；

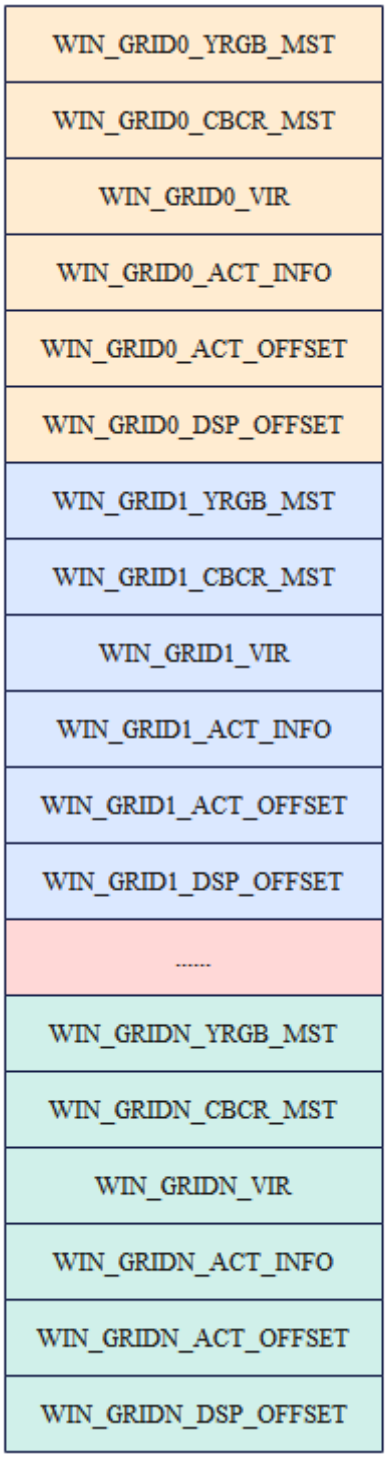
WIN_CTRL.win_num 必须设置为 0;

4.2 多宫格模式

1. 多宫格模式要配置的寄存器和单图层模式类似，包括 WIN_ACT_INFO(0x1c)，WIN_DSP_INFO(0x20)，WIN_DSP_ST(0x24)。GRID0 寄存器不再需要配置，宫格相关的寄存器通过 AXI 总线直接从 DDR 获取；
2. 使用多宫格模式时，用户需要使能 `WIN_CTRL.win_multi_grid_en` 信号，同时需要配置 `WIN_CTRL.win_num`，win_num 的配置为实际宫格数量 - 1, 即：

```
WIN_CTRL.win_num = real_num - 1, WIN_CTRL.win_multi_grid_en = 1。
```

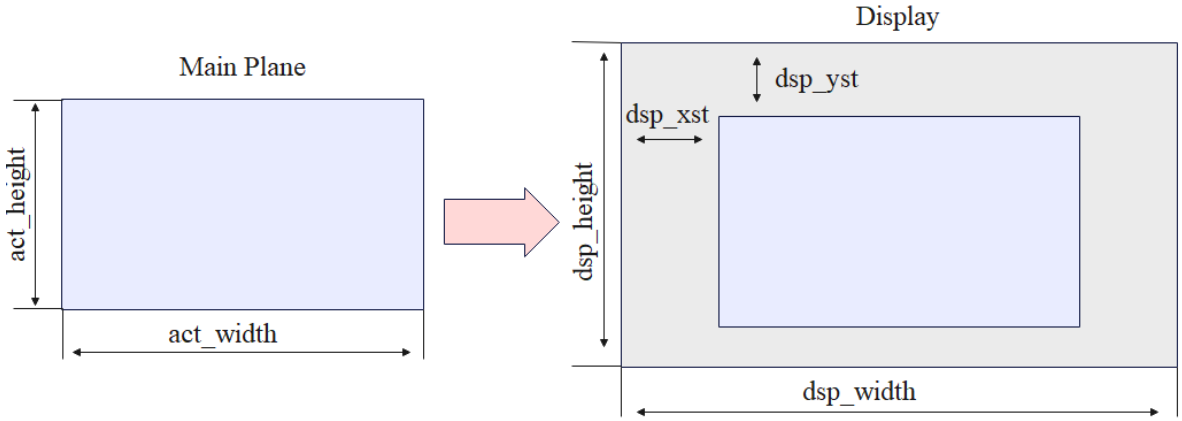
3. 多宫格的显示参数信息要按照顺序写入到 DDR (首地址要 16 Byte 对齐), 如 GRID 0~N 的写入信息如下：



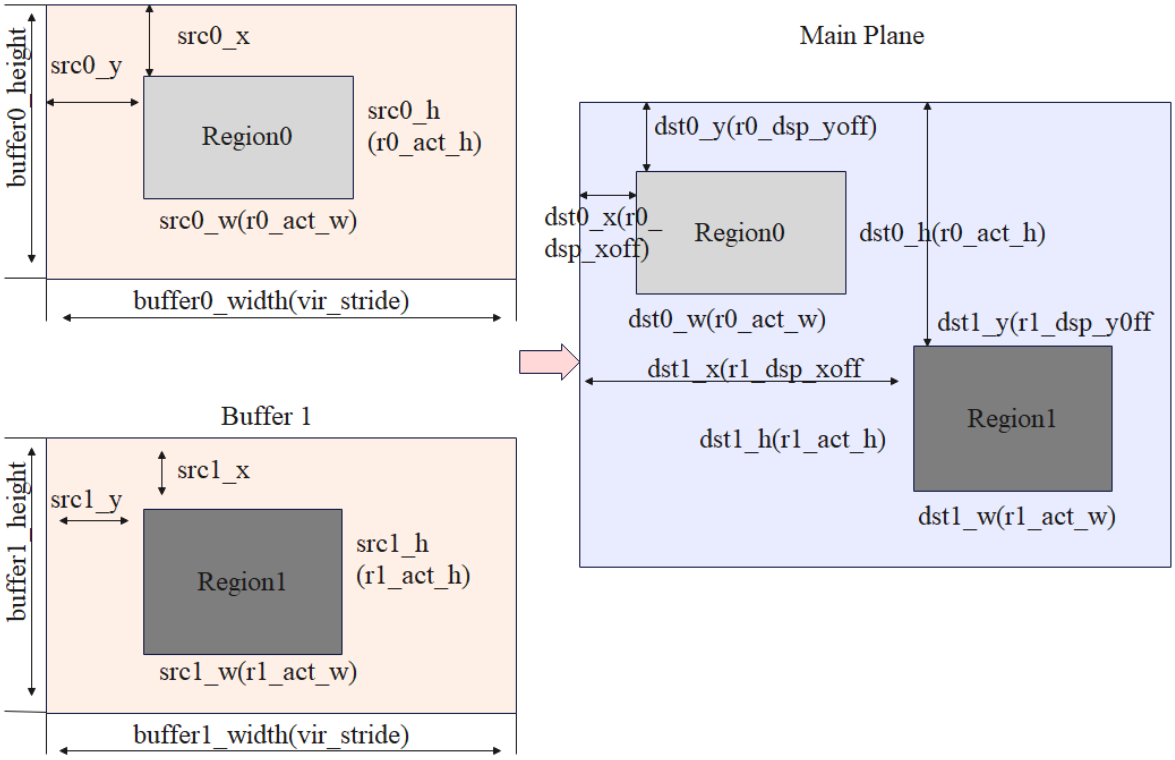
4. 多宫格的显示参数组织放入 DDR 时，有大小排序的要求，要求先按起点纵坐标从小到大，再按横坐标从小到大，进行排序；
5. msmart 图层在启动后，需要 3 个 rid(axi read id)，软件需要配置 2 个 rid 分别是 YRGB_RID、CBCR_RID, 另外的 GRID_RID 不需要软件配置，芯片硬件默认 `GRID_RID=YRGB_RID + 1`，软件的初始配置注意需要预留 GRID_RID;
6. 同单图层模式，WIN_GRIDX_ACT_OFFSET 配置设置为 0；

多宫格的参数配置可参考如下的示意图，分为整个图层的配置和内部多个子宫格的配置。

整个 msmart 图层的配置示意图：



内部多个宫格配置示意图：



4.3 驱动配置注意事项

1. 多宫格测试，宫格布局切换，第一个宫格的宽高会一直维持上电后第一个区域的宽高，更新第一个宫格的参数不起效
   问题分析：没有拉高 soft reset 可能导致内部的一些逻辑没有清干净。
   解决方案：需要保持图层的 soft reset bit 拉高。
2. 从多宫格切换到单图层，单图层显示 ARGB8888 格式会显示异常，显示 NV12 会 IOMMU 报错问题
   问题原因：多宫格切单图层的场景中，用户需要先配置 `WIN_CTRL.win_multi_grid_en = 0` 后，才能通过 CPU 对单图层模式下显示参数区域 grid0 parameters 进行写操作（0x50-0x64）。
   解决方案：软件配置调整顺序，把 `WIN_CTRL.win_multi_grid_en = 0` 的配置放在所有的 grid0 配置之前。

5. 多宫格模式功能说明

5.1 多宫格空间布局说明

如下为测试支持的部分空间布局:

5.2 多宫格图层格式说明

支持如下的 FourCC 格式：

RGB: XR30, AR30, XB30, AB30, XR24, AR24, XB24, AB24, RG24, BG24, RG16, BG16, XR15, AR15, XB15, AB15

YUV: NV12, NV21, NV16, NV61, NV24, NV42, NV15, NV20, NV30

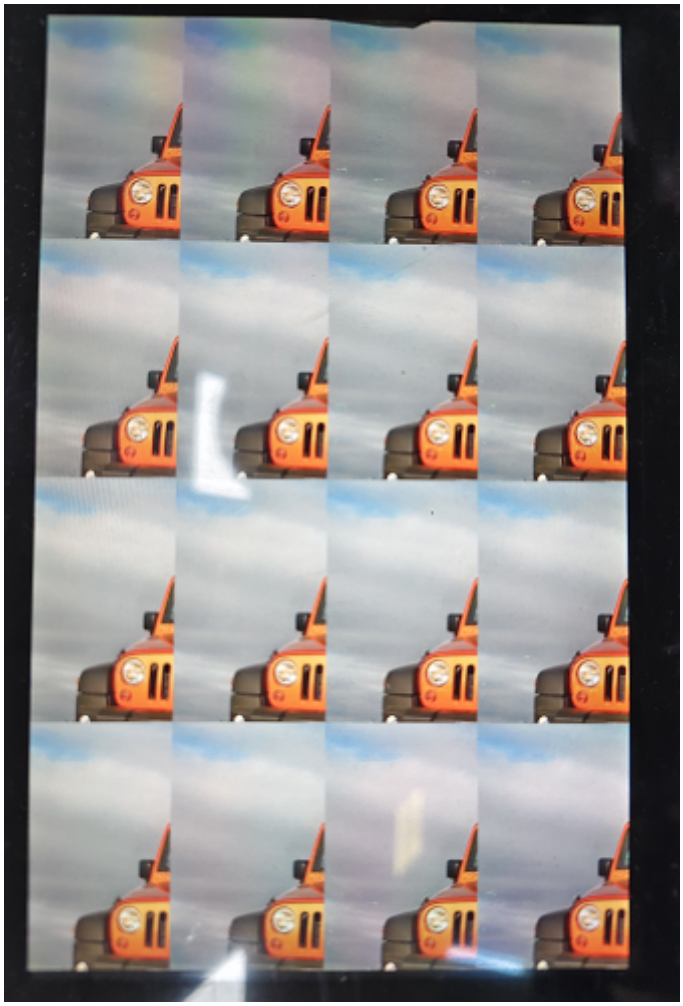
5.3 多宫格缩放和mirror说明

缩小: 缩放系数要小于 2，等于 2 会显示异常

放大: 放大倍数限制与单图层一致

缩小效果如下：

缩小前

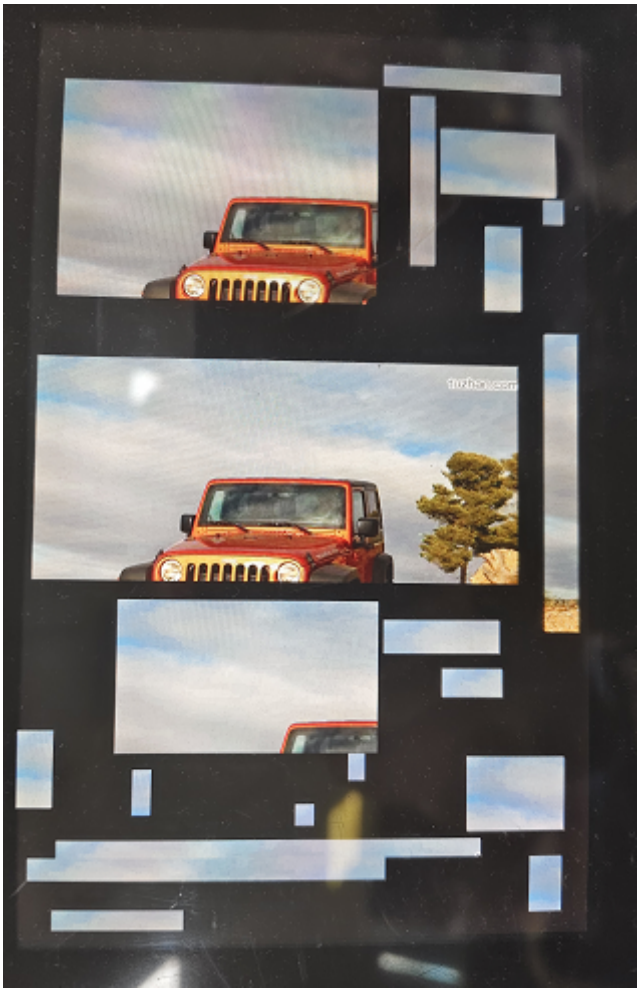


缩小后



支持 Y-Mirror，Y-Mirror 效果如下：

Y-Mirror 前



Y-Mirror 后



6. 用户空间接口说明

msmart 只注册一个 plane, 通过提供一个 msmart 图层的 blob 属性 `MSMART_DATA`, 用来传递多宫格的参数，传递参数的结构体如下：

```
#define ROCKCHIP_VOP_MSMART_MAX_GRIDS_NUM       100
#define ROCKCHIP_VOP_MSMART_LUT_LENGTH          1536

/* msmart layer grid info */
struct rockchip_vop_msmart_grid {
        u32 fb_id; /* fb object contains grid format type */

        /* dest location which should not exceed the main layer */
        u32 dst_x;
        u32 dst_y;
        u32 dst_w;
        u32 dst_h;

        /* Source values are 16.16 fixed point */
        u32 src_x;
        u32 src_y;
        u32 src_h;
        u32 src_w;

        u32 reserved[20];
};

struct msmart_data {
        u32 version;

        /* Source values are 16.16 fixed point */
        u32 src_w;
        u32 src_h;

        u32 crtc_x;
        u32 crtc_y;
        u32 crtc_w;
        u32 crtc_h;

        u32 active_grid_num;
        u32 reserved[30];
        struct rockchip_vop_msmart_grid grid[ROCKCHIP_VOP_MSMART_MAX_GRIDS_NUM];
};
```

其中，`struct msmart_data` 为配置的多宫格信息，具体信息如下：

`version`: 版本信息，当前默认为 0。

`src_w`: 主区域的宽度

`src_h`: 主区域的高度

`crtc_x`: 主区域输出显示时的 x 起始坐标

`crtc_y`: 主区域输出显示时的 y 起始坐标

`crtc_w`: 主区域输出的宽度

`crtc_h`: 主区域输出的高度

`active_grid_num`: 当前配置的多宫格的数量

`reserved`: 预留后续版本拓展使用

`grid`: 多宫格信息

`struct rockchip_vop_msmart_grid` 为具体的宫格信息：

`fb_id`: 宫格 buffer 的 fb_id

`dst_x`: 宫格在主区域上的 x 起始坐标

`dst_y`: 宫格在主区域上的 y 起始坐标

`dst_w`: 宫格在主区域上的宽度

`dst_h`: 宫格在主区域上的高度

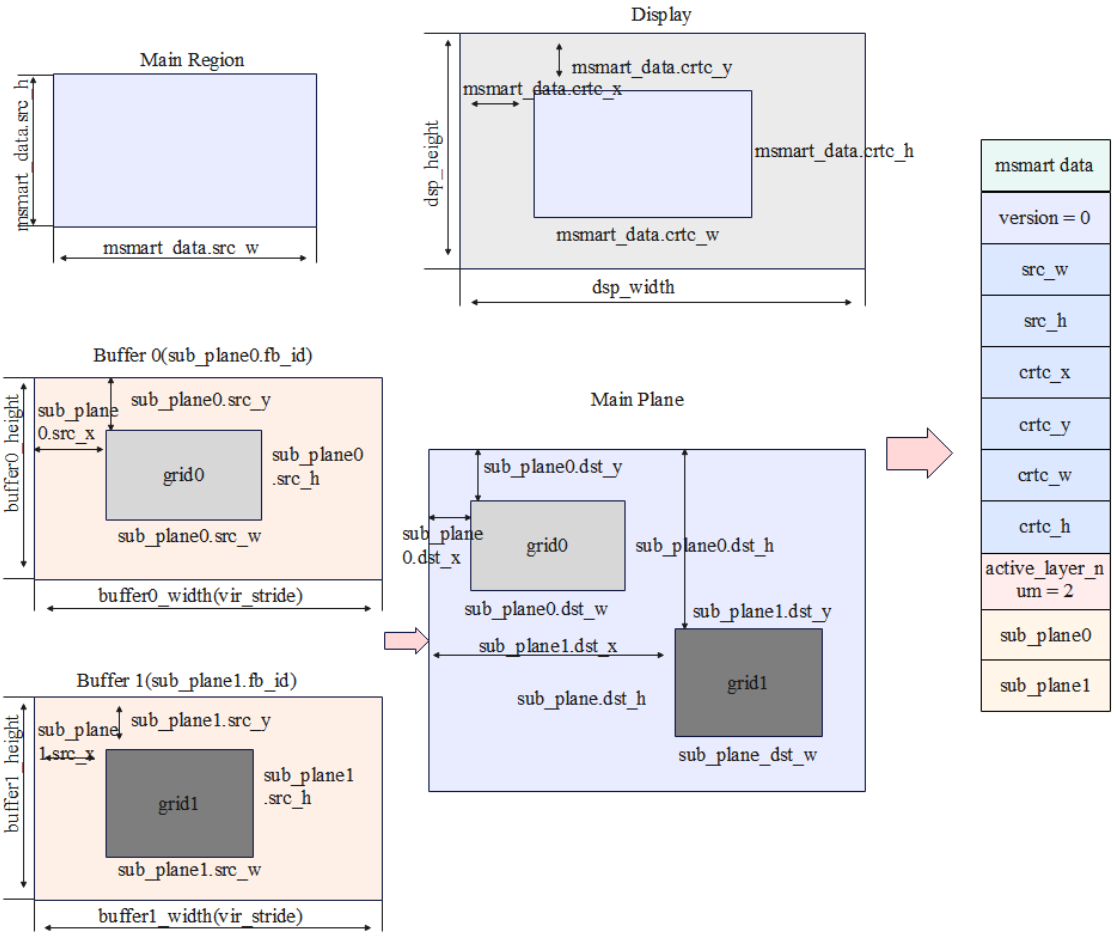
`src_x`: 宫格在 buffer 上的 x 起始坐标

`src_y`: 宫格在 buffer 上的 y 起始坐标

`src_w`: 宫格在 buffer 上的宽度

`src_h`: 宫格在 buffer 上的高度

当前上层应用使用时，把多宫格小图层的起始坐标，宽高, fb_id 等信息和整体图层的信息通过 `MSMART_DATA` 属性传递给内核驱动，同时，msmart 绑定的 drm plane 绑定的信息为第一个宫格的信息。



Note:

1. 多个宫格配置时，不要求应用根据每个宫格的起始 x, y 坐标进行排序，驱动会自动进行排序；
2. 每一帧都可以动态改变配置的宫格的数量，宫格的位置和宫格的大小；

3. 配置缩放时，只能整体缩放，即主区域的 src_w/h 和 crtc_w/h, 宫格的 src_w/h 和 crtc_w/h 需要保持一致；
4. 切换到单图层模式时，需要同时清空 `MSMART_DATA` 属性；

当前 ovltest 用法：

```
static int set_msmart_plane(struct device *dev, struct plane_arg *p,
                struct msmart_sub_plane *msmart_plane,
                const char *file_name)
{
    uint32_t handles[4] = {0}, pitches[4] = {0}, offsets[4] = {0};
    unsigned int old_fb_id;
    struct bo *plane_bo;
    char name[PIC_NAME_MAX_LEN];
    int ret = 0;

    if (!strcmp(msmart_plane->file_name, "none"))
        strcpy(name, file_name);
    else
        strcpy(name, msmart_plane->file_name);

    plane_bo = msmart_plane->old_bo;
    msmart_plane->old_bo = msmart_plane->bo;

    if (!plane_bo) {
        plane_bo = ovl_bo_create(dev->fd, p->fourcc, p->afbc_en || p->rfbc_en,
                     msmart_plane->vir_w, msmart_plane->vir_h,
                     msmart_plane->vir_h, handles, pitches, offsets, name);
        if (plane_bo == NULL)
            return -1;
        ret = drmModeAddFB2(dev->fd, msmart_plane->vir_w, msmart_plane->vir_h,
p->fourcc,
                    handles, pitches, offsets, &msmart_plane->fb_id, 0);
        if (ret) {
            fprintf(stderr, "failed to add fb: %s\n", strerror(errno));
            return -1;
        }
    }
    msmart_plane->bo = plane_bo;

    old_fb_id = p->fb_id;
    msmart_plane->old_fb_id = old_fb_id;

    return 0;
}

static int set_msmart_planes(struct device *dev, struct plane_arg *p,
                struct msmart_data *msmart_data, const char *file_name,
                unsigned int offset)
{

    int i;
    int ret = 0;
    unsigned int blob_id = 0;
    struct msmart_sub_plane *msmart_plane;

    if (!p->msmart_arg.sub_layer_num)
```

```
        return 0;

    for (i = 0; i < p->msmart_arg.sub_layer_num; i++) {
        ret = set_msmart_plane(dev, p, &p->msmart_arg.msmart_plane[i],
file_name);
        if (ret)
            return ret;
    }

    msmart_data->active_grid_num = p->msmart_arg.sub_layer_num;
    for (i = 0; i < p->msmart_arg.sub_layer_num; i++) {
        msmart_plane = &p->msmart_arg.msmart_plane[i];
        msmart_data->grid[i].fb_id = msmart_plane->fb_id;
        msmart_data->grid[i].dst_x = msmart_plane->dst_x;
        msmart_data->grid[i].dst_y = msmart_plane->dst_y;
        msmart_data->grid[i].dst_w = msmart_plane->dst_w;
        msmart_data->grid[i].dst_h = msmart_plane->dst_h;

        msmart_data->grid[i].src_x = msmart_plane->src_x << 16;
        msmart_data->grid[i].src_y = msmart_plane->src_y << 16;
        msmart_data->grid[i].src_w = msmart_plane->src_w << 16;
        msmart_data->grid[i].src_h = msmart_plane->src_h << 16;
    }
    drmModeCreatePropertyBlob(dev->fd, msmart_data, sizeof(struct msmart_data),
&blob_id);
    p->msmart_arg.blob_id = blob_id;
    add_property_optional(dev, p->plane_id, "MSMART_DATA", blob_id);

    return 0;
}

static int atomic_set_msmart_plane(struct device *dev, struct plane_arg *p,
const char *file_name,
                   bool update, unsigned int offset)
{
    struct msmart_sub_plane *msmart_plane;
    struct msmart_data msmart_data = {0};
    struct crtc *crtc = NULL;
    int crtc_x, crtc_y, crtc_w, crtc_h;
    unsigned int i;
    int ret;

    /* Find an unused plane which can be connected to our CRTC. Find the
     * CRTC index first, then iterate over available planes.
     */
    for (i = 0; i < (unsigned int)dev->resources->res->count_crtcs; i++) {
        if (p->crtc_id == dev->resources->res->crtcs[i]) {
            crtc = &dev->resources->crtcs[i];
            break;
        }
    }

    if (!crtc) {
        fprintf(stderr, "CRTC %u not found\n", p->crtc_id);
        return -1;
    }

    if (!update)
```

```
        fprintf(stderr, "testing %dx%d@%s on plane %u, crtc %u\n",
            p->w, p->h, p->format_str, p->plane_id, p->crtc_id);

    if (p->rotation & (DRM_MODE_ROTATE_90 | DRM_MODE_ROTATE_270)) {
        crtc_w = p->crtc_w ? p->crtc_h : p->h * p->scale;
        crtc_h = p->crtc_h ? p->crtc_w : p->w * p->scale;
    } else {
        crtc_w = p->crtc_w ? p->crtc_w : p->w * p->scale;
        crtc_h = p->crtc_h ? p->crtc_h : p->h * p->scale;
    }
    if (!p->has_position) {
        /* Default to the middle of the screen */
        crtc_x = (crtc->mode->hdisplay - crtc_w) / 2;
        crtc_y = (crtc->mode->vdisplay - crtc_h) / 2;
    } else {
        crtc_x = p->x;
        crtc_y = p->y;
    }
    msmart_data.src_w = p->w << 16;
    msmart_data.src_h = p->h << 16;
    msmart_data.crtc_x = crtc_x;
    msmart_data.crtc_y = crtc_y;
    msmart_data.crtc_w = crtc_w;
    msmart_data.crtc_h = crtc_h;

    ret = set_msmart_planes(dev, p, &msmart_data, file_name, offset);
    if (ret)
        return ret;

    msmart_plane = &p->msmart_arg.msmart_plane[0];

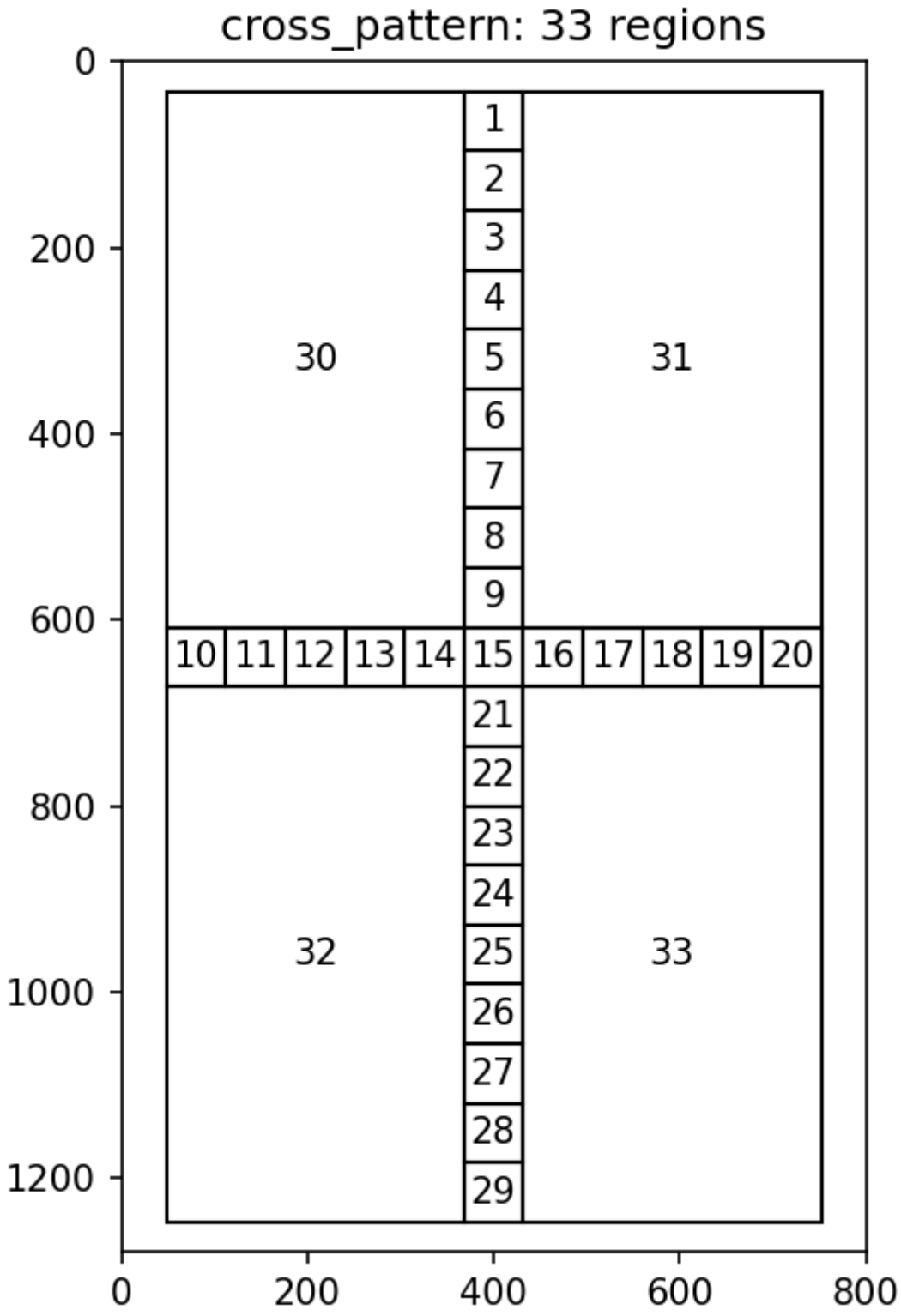
    p->old_bo = msmart_plane->old_bo;
    p->bo = msmart_plane->bo;

    add_property(dev, p->plane_id, "FB_ID", msmart_plane->fb_id);
    add_property(dev, p->plane_id, "CRTC_ID", p->crtc_id);
    add_property(dev, p->plane_id, "SRC_X", offset << 16);
    add_property(dev, p->plane_id, "SRC_Y", offset << 16);
    add_property(dev, p->plane_id, "SRC_W", (msmart_plane->src_w - offset) <<
16);
    add_property(dev, p->plane_id, "SRC_H", (msmart_plane->src_h - offset) <<
16);
    add_property(dev, p->plane_id, "CRTC_X", msmart_plane->dst_x);
    add_property(dev, p->plane_id, "CRTC_Y", msmart_plane->dst_y);
    add_property(dev, p->plane_id, "CRTC_W", msmart_plane->dst_w);
    add_property(dev, p->plane_id, "CRTC_H", msmart_plane->dst_h);
    add_property(dev, p->plane_id, "rotation", p->rotation);
    add_property(dev, p->plane_id, "zpos", p->zpos);
    if (p->no_premulti)
        add_property(dev, p->plane_id, "pixel blend mode", 1);
    else
        add_property(dev, p->plane_id, "pixel blend mode", 0);
    add_property(dev, p->plane_id, "EOTF", p->eotf);
    add_property(dev, p->plane_id, "COLOR_SPACE", p->colorspace);

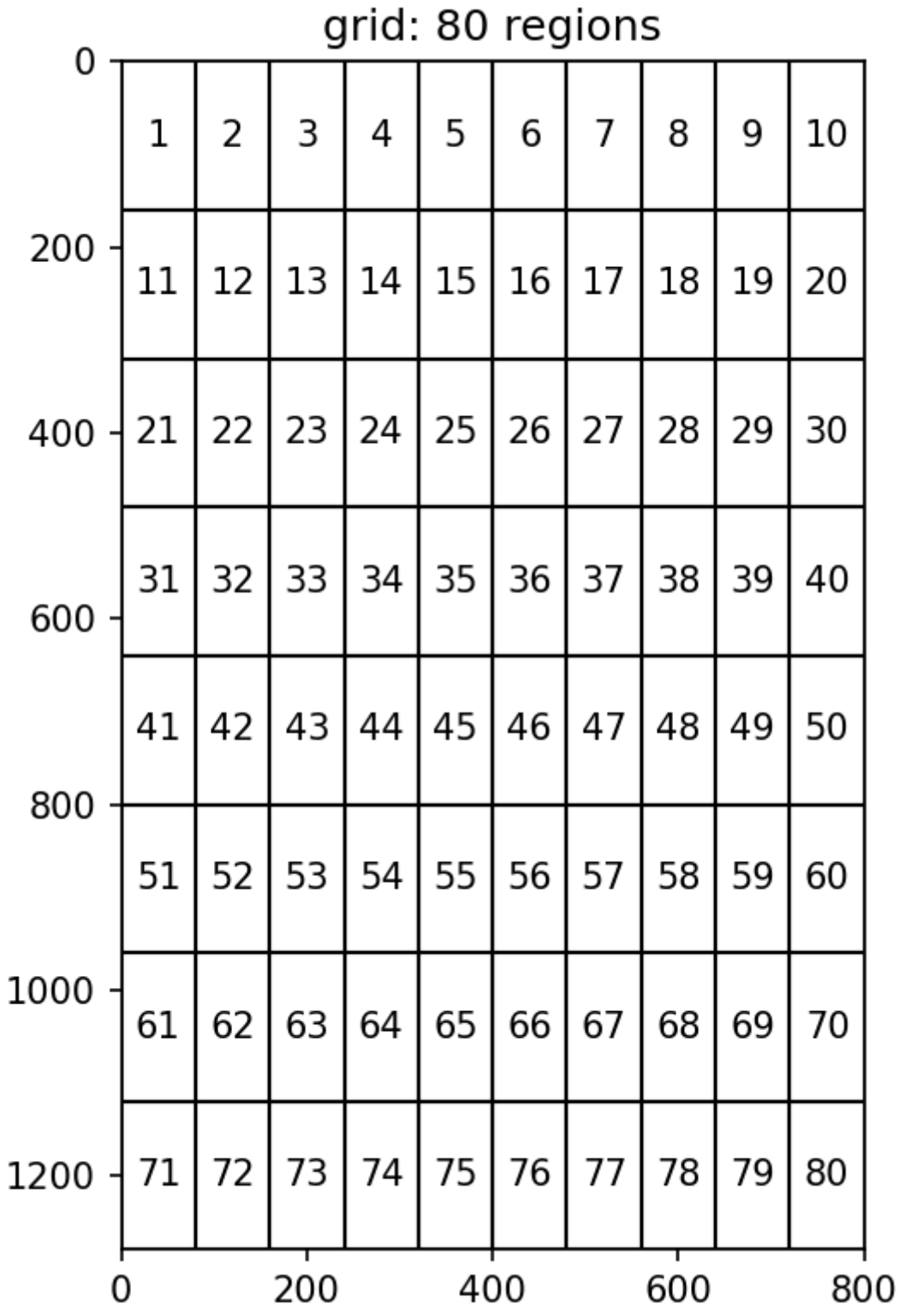
    return 0;
}
```

msmart 图层不支持的 case:

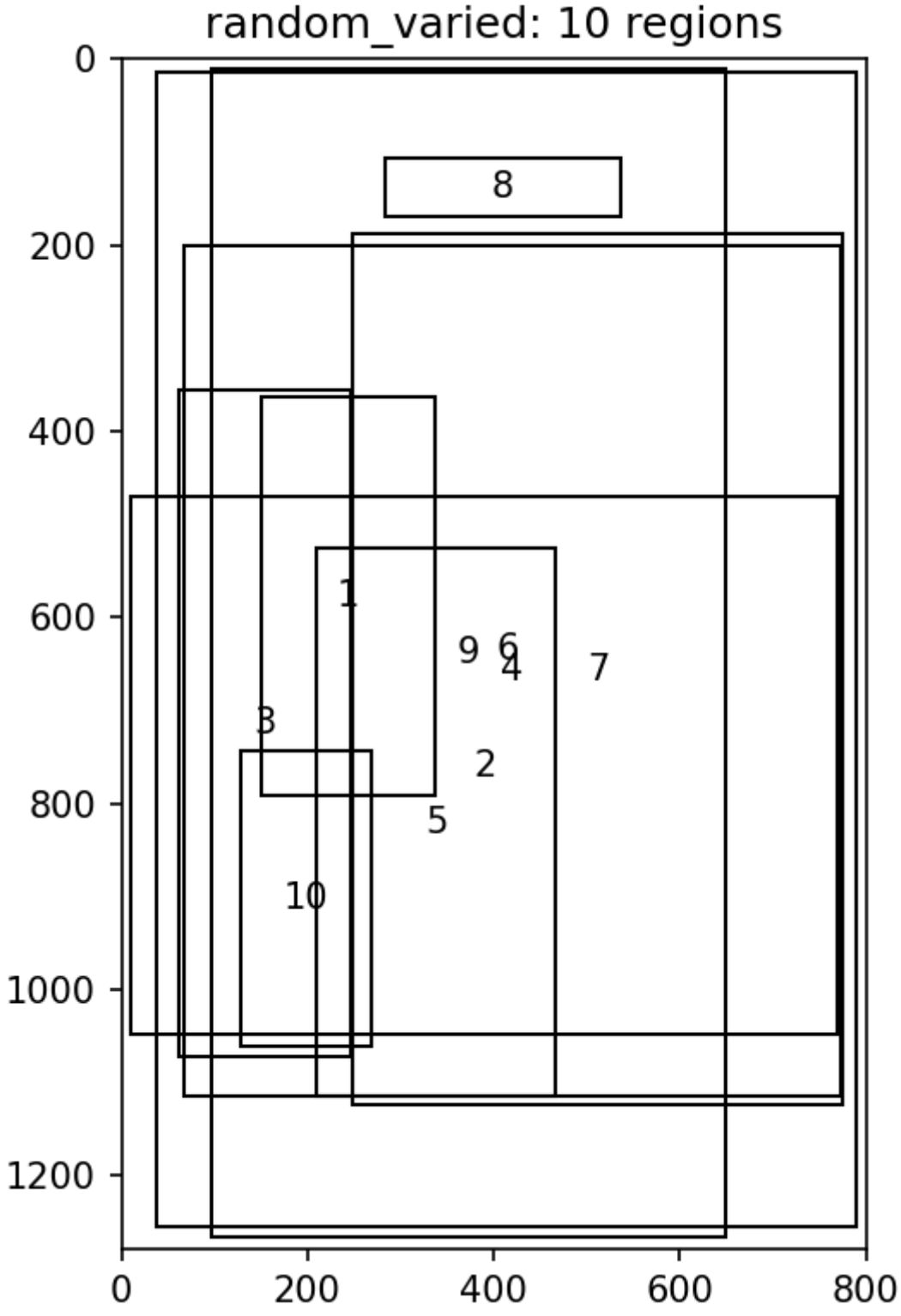
超出一行最大宫格限制：



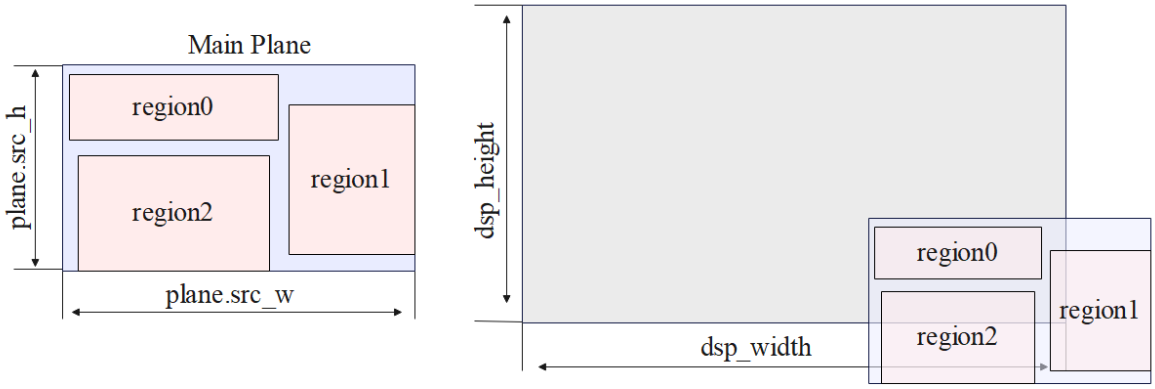
超出一行最大宫格限制并且超出最大数量限制：


grid: 80 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80

宫格重叠：


random_varied: 10 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
8
1
9
6
4
7
3
2
5
10

超出屏幕显示范围:


Main Plane
region0
region1
region2
plane.src_h
plane.src_w
dsp_height
dsp_width
region0
region1
region2